

AI Medical Record Intelligence Platform — Complete Roadmap

Project overview

Build a full-stack AI-powered medical record platform where users upload medical reports (PDFs, images), and the system extracts data using OCR, understands it using NLP, predicts disease risks using ML, and allows conversational Q&A using RAG + LLM. Target stack: Next.js frontend, FastAPI backend, PostgreSQL + FAISS database, Docker deployment.

Phase 1 — Data collection & preparation

Prompt to use:

"I am building an AI medical record platform. Help me find and prepare datasets for: (1) diabetes prediction, (2) heart disease prediction, (3) medical NLP entity extraction. I need datasets from Kaggle or UCI ML Repository. Also suggest how to augment data with synthetic medical reports for OCR training."

Datasets to collect:

- Pima Indians Diabetes Dataset (Kaggle/UCI)
- Cleveland Heart Disease Dataset (UCI)
- MIMIC-III clinical notes (PhysioNet — for NLP)
- Custom: create 20–30 sample PDF medical reports manually for OCR testing

Data preparation steps:

1. Download CSVs, inspect with `df.info()` and `df.describe()`
 2. Check for null values, outliers, class imbalance
 3. Save raw data in `data/raw/`, cleaned data in `data/processed/`
 4. Version your data with DVC or at minimum a `data_version.txt` file
-

Phase 2 — Exploratory data analysis (EDA)

Prompt to use:

"I have the Pima Indians Diabetes dataset with columns: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, Outcome. Perform full EDA — show distribution plots, correlation heatmap, class balance check, outlier detection using IQR, and suggest which features are most predictive."

EDA tasks:

1. Plot histograms for all numerical features using seaborn
 2. Plot correlation heatmap with `sns.heatmap(df.corr())`
 3. Check class balance: `df['Outcome'].value_counts()`
 4. Identify outliers using boxplots and IQR method
 5. Save all EDA plots to notebooks/eda/ folder
-

Phase 3 — Feature engineering

Prompt to use:

"For the diabetes prediction dataset, suggest feature engineering techniques. I want to: handle zero values in Glucose/BMI/BloodPressure (which are biologically impossible), create BMI category bins (Underweight/Normal/Overweight/Obese), create an age group feature, apply log transformation to skewed features like Insulin, and use SMOTE for class imbalance. Show me the full Python code."

Feature engineering steps:

1. Replace zero values in biological columns with median: `df['Glucose'].replace(0, df['Glucose'].median(), inplace=True)`
 2. Create BMI categories using `pd.cut()`
 3. Create age groups: young/middle/senior
 4. Apply `np.log1p()` to skewed columns (Insulin, SkinThickness)
 5. Apply SMOTE: `from imblearn.over_sampling import SMOTE`
 6. Scale features: StandardScaler for linear models, skip for tree models
 7. Save the fitted scaler with `joblib.dump(scaler, 'models/scaler.pkl')`
-

Phase 4 — Model building & evaluation

Prompt to use:

"I am building a diabetes risk prediction model using scikit-learn. Train and compare: Logistic Regression, Random Forest, XGBoost, and a simple Neural Network using PyTorch. Evaluate using: accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix. Then explain how to add SHAP explainability so the model can tell the user WHY it flagged them as high risk."

Model pipeline:

```
python

# models to train

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

models = {
    "LogisticRegression": LogisticRegression(),
    "RandomForest": RandomForestClassifier(n_estimators=100),
    "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric='logloss')
}

# evaluate each

from sklearn.metrics import classification_report, roc_auc_score
for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    print(f"\n{name}")
    print(classification_report(y_test, preds))
    print("ROC-AUC:", roc_auc_score(y_test, model.predict_proba(X_test)[: ,1]))
```

SHAP explainability (the FAANG differentiator):

```
python

import shap

explainer = shap.TreeExplainer(best_model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)

# For a single patient prediction:
```

```
shap.force_plot(explainer.expected_value, shap_values[0], X_test.iloc[0])
```

```
Save best model: joblib.dump(best_model, 'models/diabetes_model.pkl')
```

Phase 5 — OCR pipeline

Prompt to use:

"I need to extract text from uploaded medical PDF reports and images using Python. Use EasyOCR and also handle PDFs using PyMuPDF (fitz). Show me a complete function that: accepts a file path, detects if it's PDF or image, extracts all text, cleans the text (remove extra whitespace, fix common OCR errors), and returns a clean string. Also show how to extract specific values like 'Glucose: 240 mg/dL' using regex."

OCR pipeline code structure:

```
python
```

```
# backend/services/ocr_service.py
```

```
import fitz # PyMuPDF
```

```
import easyocr
```

```
import re
```

```
reader = easyocr.Reader(['en'])
```

```
def extract_text(file_path: str) -> str:
```

```
    if file_path.endswith('.pdf'):
```

```
        return extract_from_pdf(file_path)
```

```
    else:
```

```
        return extract_from_image(file_path)
```

```
def extract_from_pdf(path):
```

```
    doc = fitz.open(path)
```

```
    text = ""
```

```

for page in doc:
    text += page.get_text()
return clean_text(text)

def extract_from_image(path):
    results = reader.readtext(path, detail=0)
    return clean_text(" ".join(results))

def clean_text(text):
    text = re.sub(r'\s+', ' ', text)
    return text.strip()

def extract_medical_values(text):
    patterns = {
        'glucose': r'[Gg]lucose[:\s]+(\d+\.\d*)',
        'hba1c': r'HbA1c[:\s]+(\d+\.\d*)',
        'bp': r'[Bb]lood\s[Pp]ressure[:\s]+(\d+/\d+)',
    }
    values = {}
    for key, pattern in patterns.items():
        match = re.search(pattern, text)
        if match:
            values[key] = match.group(1)
    return values

```

Phase 6 — NLP & medical entity extraction

Prompt to use:

"I have extracted raw text from a medical blood test report. I want to use spaCy and a BioBERT model to identify: disease names, medicine names, abnormal lab values, symptoms. Show me how to load a pre-trained BioBERT NER model from HuggingFace, run inference on a medical text string, and return a structured JSON of extracted entities with their types and values."

NLP service structure:

```
python
```

```
# backend/services/nlp_service.py
```

```
from transformers import pipeline
```

```
ner_pipeline = pipeline(  
    "ner",  
    model="allenai/scibert_scivocab_uncased",  
    aggregation_strategy="simple"  
)
```

```
def extract_entities(text: str) -> dict:  
    entities = ner_pipeline(text)  
    result = {"diseases": [], "medicines": [], "values": []}  
    for ent in entities:  
        if ent['entity_group'] in ['DISEASE', 'CONDITION']:  
            result['diseases'].append(ent['word'])  
        elif ent['entity_group'] in ['CHEMICAL', 'DRUG']:  
            result['medicines'].append(ent['word'])  
    return result
```

Phase 7 — RAG chatbot (the WOW feature)

Prompt to use:

"I want to build a RAG (Retrieval Augmented Generation) chatbot using LangChain where users can ask questions about their uploaded medical reports. The flow is: (1) user uploads report → text is extracted and split into chunks → chunks are embedded using sentence-transformers → embeddings stored in FAISS vector DB. (2) User asks a question → question is embedded → top-k similar chunks retrieved from FAISS → chunks + question sent to Claude/GPT API → answer returned. Show me the complete LangChain Python code for both the indexing pipeline and the query pipeline."

RAG pipeline:

python

```
# backend/services/rag_service.py
```

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
from langchain.embeddings import HuggingFaceEmbeddings
```

```
from langchain.vectorstores import FAISS
```

```
from langchain.chains import RetrievalQA
```

```
from langchain.llms import Anthropic # or OpenAI
```

```
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-  
MiniLM-L6-v2")
```

```
def index_report(text: str, user_id: str):
```

```
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
```

```
    chunks = splitter.split_text(text)
```

```
    db = FAISS.from_texts(chunks, embeddings)
```

```
    db.save_local(f"vector_stores/{user_id}")
```

```
    return len(chunks)
```

```
def query_report(question: str, user_id: str) -> str:
```

```
    db = FAISS.load_local(f"vector_stores/{user_id}", embeddings)
```

```
    retriever = db.as_retriever(search_kwargs={"k": 4})
```

```
qa_chain = RetrievalQA.from_chain_type(
    llm=Anthropic(model="claude-3-sonnet"),
    retriever=retriever
)

return qa_chain.run(question)
```

Phase 8 — Folder structure

Prompt to use:

"Show me the complete production-ready folder structure for a full-stack AI app with: Next.js 14 (App Router) frontend, FastAPI Python backend, PostgreSQL database, FAISS vector store, ML models served via FastAPI, Docker Compose for orchestration. Explain the purpose of each folder."

medical-ai-platform/

```
|
|
| └─ frontend/           # Next.js 14 app
|   |
|   | └─ app/
|   |   |
|   |   | └─ (auth)/
|   |   |   |
|   |   |   | └─ login/page.tsx
|   |   |   |   └─ register/page.tsx
|   |   |   └─ dashboard/page.tsx  # main user dashboard
|   |   |   └─ upload/page.tsx    # report upload page
|   |   |   └─ chat/page.tsx     # RAG chatbot UI
|   |   |   └─ reports/[id]/page.tsx  # individual report view
|   |   |   └─ layout.tsx
|   |   └─ components/
|   |       └─ ui/           # reusable components
|   |       └─ ReportCard.tsx
|   |       └─ ChatWindow.tsx
```

```

| | └─ RiskChart.tsx      # SHAP chart visualization
| | └─ UploadZone.tsx
| └─ lib/
| | └─ api.ts            # axios API calls to FastAPI
| | └─ auth.ts
| └─ types/
| | └─ index.ts          # TypeScript interfaces
| └─ Dockerfile
| └─ package.json
|
└─ backend/              # FastAPI Python app
  └─ app/
    └─ main.py           # FastAPI app entry point
    └─ routers/
      └─ auth.py         # /auth/login, /auth/register
      └─ reports.py      # /reports/upload, /reports/{id}
      └─ predict.py      # /predict/diabetes, /predict/heart
      └─ chat.py         # /chat/query (RAG endpoint)
      └─ services/
        └─ ocr_service.py # EasyOCR + PyMuPDF
        └─ nlp_service.py # BioBERT NER
        └─ rag_service.py # LangChain + FAISS
        └─ predict_service.py # ML model inference + SHAP
      └─ models/
        └─ user.py       # SQLAlchemy User model
        └─ report.py     # SQLAlchemy Report model
      └─ schemas/

```

```

| | | ├── user.py      # Pydantic schemas
| | | └── report.py
| | ├── database.py    # PostgreSQL connection
| | └── config.py      # env vars with pydantic-settings
| ├── ml_models/       # saved .pkl and .pt files
| | ├── diabetes_model.pkl
| | ├── heart_model.pkl
| | └── scaler.pkl
| ├── vector_stores/   # FAISS indexes per user
| ├── uploads/         # temporary uploaded files
| ├── Dockerfile
| └── requirements.txt
|
|── data/
| ├── raw/             # original downloaded datasets
| ├── processed/       # cleaned, feature-engineered CSVs
| └── sample_reports/  # test PDFs and images
|
|── notebooks/
| ├── eda/             # EDA notebooks
| ├── model_training/  # model training notebooks
| └── experiments/     # experiment tracking
|
|── docker-compose.yml  # orchestrates all services
|── .env               # environment variables
|── .env.example
└── README.md

```

Phase 9 — FastAPI backend setup

Prompt to use:

"Show me how to set up a FastAPI app with: SQLAlchemy ORM connected to PostgreSQL, JWT authentication with bcrypt password hashing, file upload endpoint that accepts PDF and image files and saves them to disk, CORS middleware configured for Next.js on localhost:3000, and a health check endpoint. Include the main.py, database.py, and auth router files."

Key main.py structure:

```
python
```

```
from fastapi import FastAPI
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
from app.routers import auth, reports, predict, chat
```

```
from app.database import engine, Base
```

```
Base.metadata.create_all(bind=engine)
```

```
app = FastAPI(title="Medical AI Platform")
```

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
app.include_router(auth.router, prefix="/auth")
```

```
app.include_router(reports.router, prefix="/reports")
```

```
app.include_router(predict.router, prefix="/predict")
app.include_router(chat.router, prefix="/chat")
```

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

Phase 10 — Next.js frontend setup

Prompt to use:

"Set up a Next.js 14 app with App Router for a medical AI platform. I need: TypeScript, Tailwind CSS, shadcn/ui components, axios for API calls to a FastAPI backend at <http://localhost:8000>, NextAuth.js for JWT authentication, React Query (TanStack Query) for server state management, a drag-and-drop file upload component using react-dropzone, and a chat UI component that streams responses. Show me the project setup commands and the key component files."

Setup commands:

```
bash

npx create-next-app@latest frontend --typescript --tailwind --app
cd frontend

npx shadcn-ui@latest init

npm install axios @tanstack/react-query react-dropzone next-auth
```

Phase 11 — Docker setup

Prompt to use:

"Write a complete docker-compose.yml for a full-stack app with 4 services: (1) next-frontend: Next.js app on port 3000, (2) fastapi-backend: FastAPI app on port 8000, (3) postgres: PostgreSQL 15 with a named volume, (4) redis: Redis for caching (optional). Also write the Dockerfile for the FastAPI service using a Python 3.11 slim base image with multi-stage build to keep image size small. And the Dockerfile for Next.js using node:18-alpine. Include health checks and depends_on."

docker-compose.yml:

yaml

version: '3.9'

services:

frontend:

build: ./frontend

ports:

- "3000:3000"

environment:

- NEXT_PUBLIC_API_URL=http://backend:8000

depends_on:

- backend

backend:

build: ./backend

ports:

- "8000:8000"

environment:

- DATABASE_URL=postgresql://user:password@postgres:5432/medicaldb

- SECRET_KEY=your-secret-key

volumes:

- ./backend/uploads:/app/uploads

- ./backend/vector_stores:/app/vector_stores

- ./backend/ml_models:/app/ml_models

depends_on:

postgres:

condition: service_healthy

postgres:

image: postgres:15

environment:

POSTGRES_USER: user

POSTGRES_PASSWORD: password

POSTGRES_DB: medicaldb

volumes:

- postgres_data:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U user -d medicaldb"]

interval: 5s

timeout: 5s

retries: 5

volumes:

postgres_data:

backend/Dockerfile:

dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

frontend/Dockerfile:

dockerfile

FROM node:18-alpine

WORKDIR /app

COPY package*.json .

RUN npm install

COPY . .

RUN npm run build

CMD ["npm", "start"]

Run everything:

bash

docker-compose up --build

Phase 12 — GitHub Actions CI/CD**Prompt to use:**

"Write a GitHub Actions workflow file that: on push to main branch, (1) runs Python tests using pytest for the FastAPI backend, (2) runs Next.js build to check for errors, (3) builds both Docker images, (4) pushes them to Docker Hub, (5) SSHs into a Render/Railway/EC2 server and runs docker-compose pull && docker-compose up -d. Include environment secrets for DOCKER_USERNAME, DOCKER_PASSWORD, and SERVER_SSH_KEY."

Phase 13 — README & portfolio presentation**Prompt to use:**

"Write a professional GitHub README for an AI Medical Record Intelligence Platform built with Next.js, FastAPI, PostgreSQL, FAISS, LangChain, and scikit-learn. Include: a

one-paragraph project summary, a features list, a system architecture diagram description, tech stack table, setup instructions (local and Docker), API endpoint table, ML model performance metrics table (accuracy/F1/AUC), screenshots section, and a 'what I learned' section written for a fresher targeting FAANG companies. Make it impressive but honest."

Interview talking points (memorize these)

When a FAANG interviewer asks about this project, use these answers:

"Why did you add SHAP?"

"Most students just output a prediction percentage. I wanted the system to be explainable — so a doctor or patient can see that 'high glucose contributed 67% to this risk flag'. That's the difference between a toy model and a production medical AI."

"How does your RAG pipeline work?"

"The uploaded report text is chunked into 500-token segments, embedded using sentence-transformers, and stored in a FAISS vector index per user. At query time, the question is embedded, top-4 similar chunks are retrieved, and those chunks plus the question are sent to the LLM as context. The LLM never hallucinates facts because it only answers from the actual report content."

"How would you scale this?"

"Replace FAISS with Pinecone for distributed vector search, move ML inference to a separate microservice with a job queue (Celery + Redis), add horizontal scaling behind a load balancer, and use S3 for file storage instead of local disk."

Copy any phase prompt directly into ChatGPT, Gemini, or paste it to a senior dev on Discord/LinkedIn. Each prompt is self-contained so you don't need to share the whole document every time.